



9. Stacks

Last In, First Out — What Goes on Top Comes Off First

Previous Section:

 [8. Linked Lists](#)

Contents:

[1. What is a Stack?](#)

[2. Types of Stacks](#)

[3. Applications of Stacks](#)

[Advantages of Stacks](#)

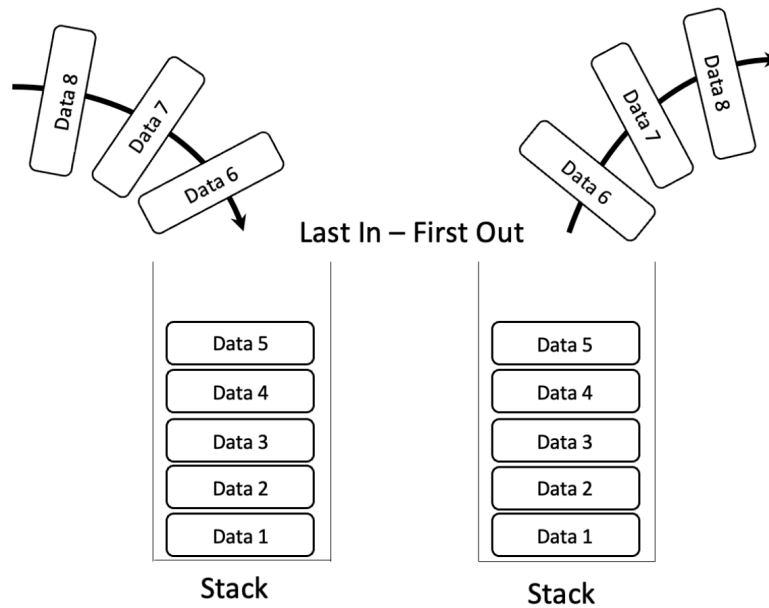
[Disadvantages of Stacks](#)

[Summary](#)

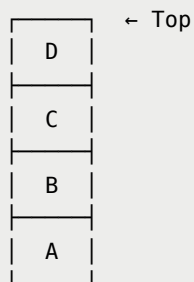
[References](#)

Stacks are one of the most common data structures we encounter, both in computer science and in everyday life.

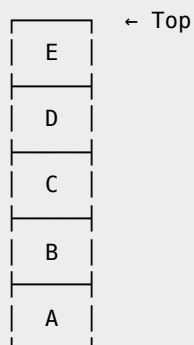
Think about a stack of books, a stack of plates, a stack of pancakes, or even a pile of cards. Whenever we add a new item, we place it on the top. And when we want to remove something, we naturally take the top item first.



If we add a new element (like **E**), it becomes the new top. And if we remove an element, that same element E will be removed first because it was the last element added. This simple idea forms the foundation of the **Stack** data structure.



If we add a new element E, it becomes the new top:



1. What is a Stack?

A **Stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle. This means that the most recently inserted element is always the first one removed.

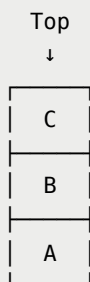
In a stack:

- New elements are inserted at the **top**
- Existing elements are removed from the **top**
- Only the **top element** can be directly accessed

Because of this restriction, we can only interact with one end of the stack.

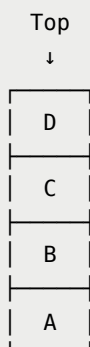
Push Operation

Before:



Push D

After:



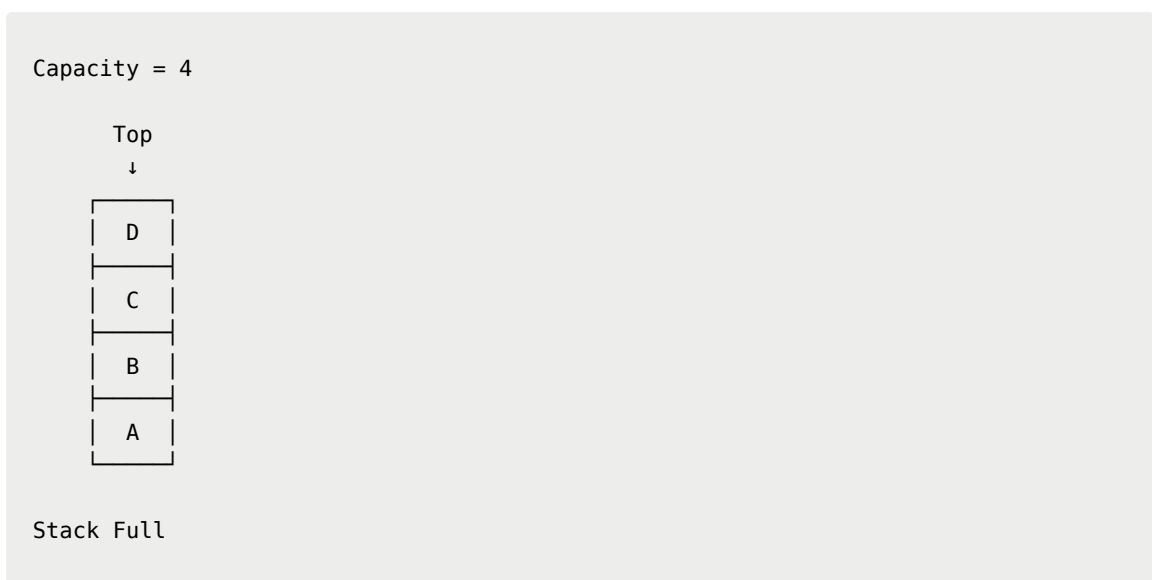
Stacks do not exist as a special built-in structure in most programming languages. Instead, we usually implement them using: Arrays; Python Lists;

Linked Lists. No matter which implementation we choose, we must always maintain a **Top Pointer**, which keeps track of the most recently inserted element.

2. Types of Stacks

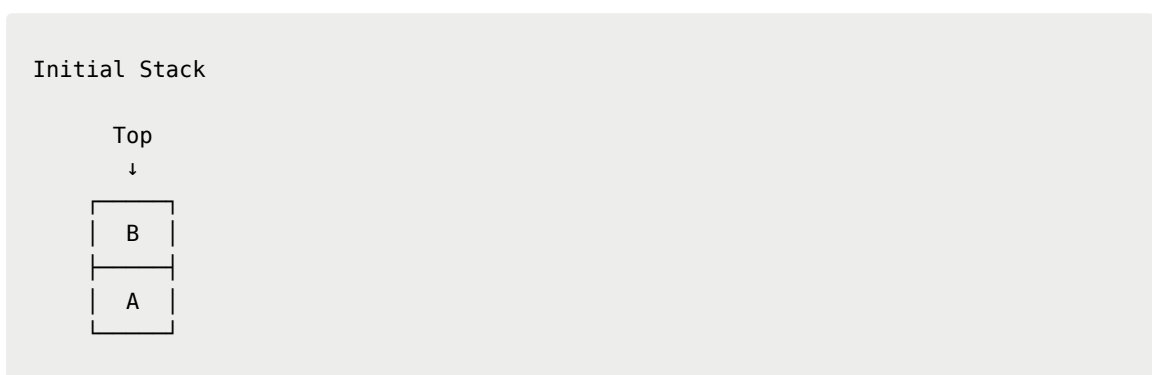
Stacks come in two common forms depending on how their storage is managed.

- **Fixed Size Stack:** A fixed-size stack has a predefined capacity. Once the stack becomes full, no more elements can be inserted.



Attempting to insert another element results in an **Overflow Error**. Likewise, attempting to remove an element from an empty stack results in an **Underflow Error**.

- **Dynamic Size Stack:** A dynamic stack can grow or shrink as needed. Instead of having a fixed capacity, new storage is allocated whenever additional space is required.



After More Push Operations



Dynamic stacks are commonly implemented using **Linked Lists**, making it easy to expand or reduce the stack during execution.

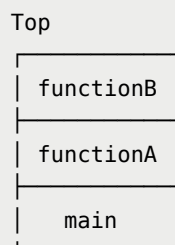
3. Applications of Stacks

Although stacks are simple, they are used extensively in software development and computer systems.

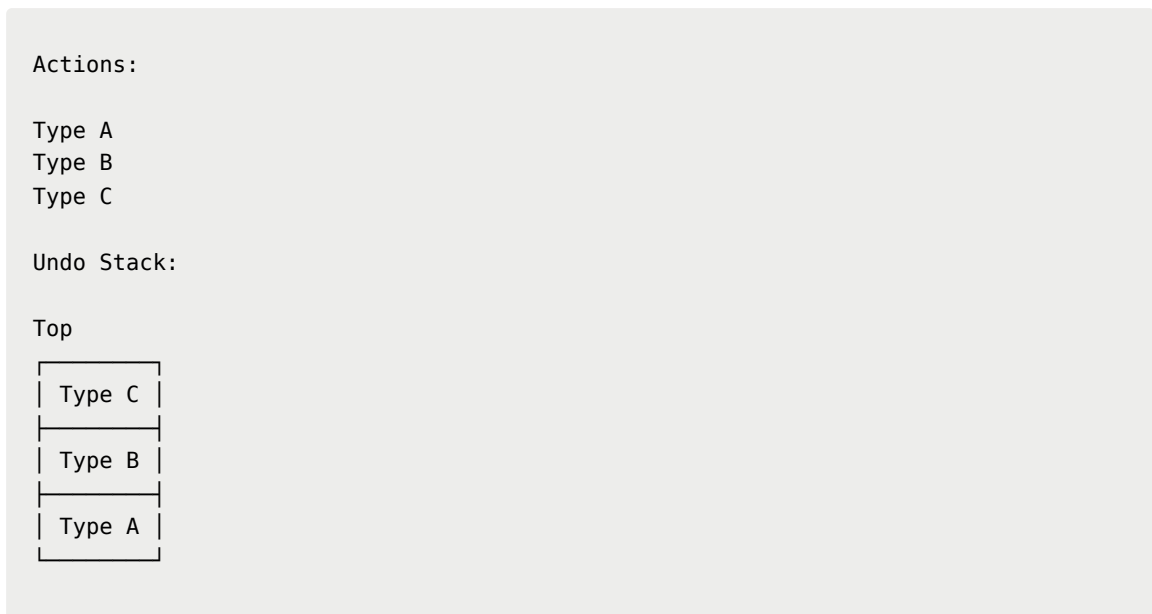
- **Function Calls:** Whenever a function is called, information about that function is pushed onto a stack. When the function finishes execution, that information is popped from the stack, allowing the program to return to the correct location.

```
main()
  ↓
functionA()
  ↓
functionB()
```

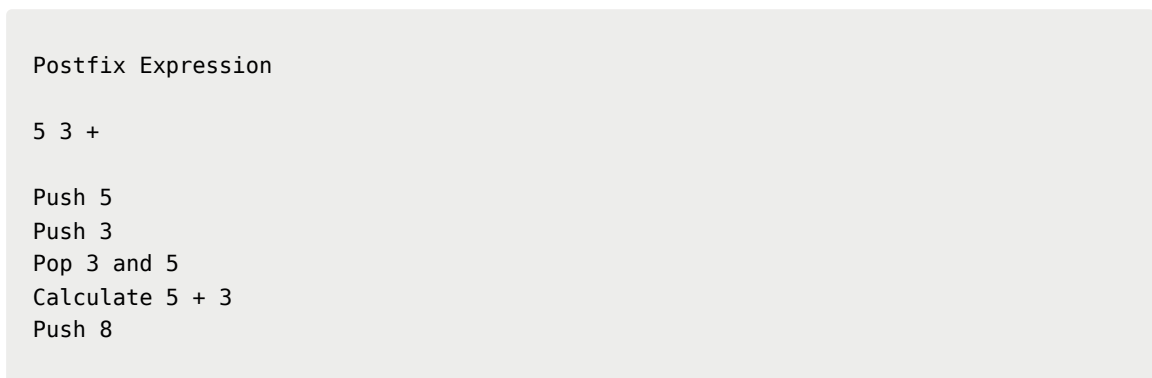
Stack:



- **Undo Operations:** Most text editors use stacks to implement the Undo feature. Each modification is pushed onto a stack. When we press Undo, the most recent action is removed first.



- **Recursion:** Recursive functions rely heavily on stacks. Each recursive call creates a new stack frame containing local variables and return information.
- **Expression Evaluation:** Stacks are frequently used when evaluating Infix expressions; Prefix expressions; Postfix expressions.



- **Syntax Parsing:** Compilers and interpreters use stacks to verify matching symbols.

```
{ [ ( ) ] }
```

```
Push {  
Push [  
Push (  
Pop (  
Pop [  
Pop {
```

Valid Syntax

- **Memory Management:** Many operating systems and programming languages use stacks to manage temporary memory allocations and execution contexts.

Advantages of Stacks

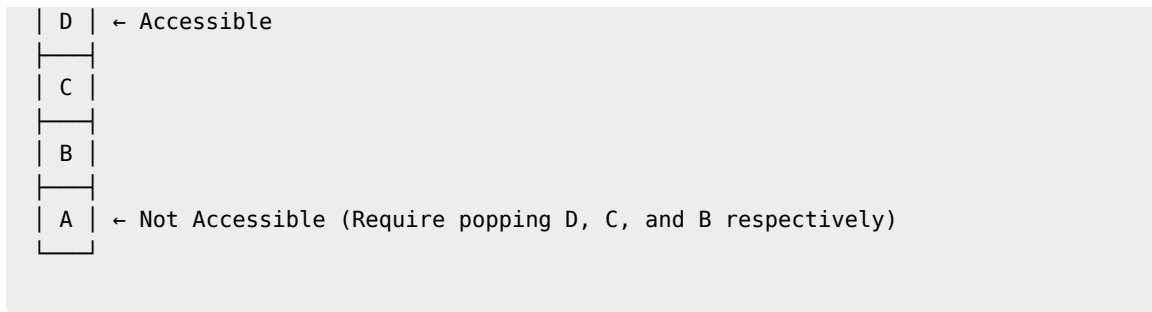
- **Fast Operations:** Push and Pop operations can be performed in constant time. Both Push and Pop operation only takes time complexity $O(1)$. This makes stacks highly efficient.
- **Simple Structure:** Stacks are easy to understand, implement, and maintain.
- **Natural LIFO Behavior:** Many real-world problems naturally follow the Last In, First Out principle. Examples include Function calls; Undo operations; Browser history; Recursion
- **Memory Efficient:** Stacks only store the elements currently needed, reducing unnecessary memory usage.

Disadvantages of Stacks

- **Limited Access:** We can only access the element at the top. For instance, to access **A which at the bottom of the stack**, we must first remove **D, C, and B**.



```
Top  
↓  
└───┘
```



- **Overflow Risk:** In fixed-size stacks, adding too many elements causes overflow.
- **No Random Access:** Unlike arrays, stacks do not allow direct access to arbitrary elements. Finding an element near the bottom may require removing many elements first.
- **Capacity Limitations:** Fixed-size stacks may become problematic when the required storage size is unknown beforehand.

Summary

A **Stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle, where the most recently inserted element is always the first one removed.

We interact with a stack through a single end called the **Top**, making insertion and deletion simple and efficient. Stacks can be implemented using arrays, Python lists, or linked lists, and may be either fixed-size or dynamically sized.

Because of their simplicity and efficiency, stacks are widely used in function calls, recursion, expression evaluation, syntax checking, undo operations, and memory management. While they provide extremely fast insertion and deletion operations, they also restrict access to only the top element, making random access impossible.

In the next section, we will explore the **common operations of a Stack**, including **Push, Pop, Peek, Search, and Traversal**, and learn how to implement them in Python.

References

https://drive.google.com/file/d/1yBJEh1C9V-nXZ7TF80JtWvozfaDYRq5B/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/stack-data-structure/>

Next Sections:

 [9.1. Operations on Stacks](#)

 [9.2. Expression Evaluation using Stacks](#)

 [9.3. Converting Infix Expressions using Stacks](#)